MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Temciuc Adelina, FAF-242

Report

on laboratory works no. 7 and 8
Operating Systems
Topic: Processes, Redirection and Pipes on Linux and in xv6

Checked by:
Reiţman Patricia, university assistant
DISA, FCIM, UTM

Chișinău – 2026

AIM OF THE WORK

Purpose of the laboratory work

The purpose of these two laboratory works is to see how a process is born, where a program writes its output and how two programs are connected with a pipe. In Lab 7 this was done on a real Linux system with three short C programs that repeat what the shell does for `ls -l`, `ls -l > out.txt` and `ls /etc | wc -l`. In Lab 8 the same system calls were used inside xv6, the small teaching operating system, to show that they are the same, and two processes were made to talk through pipes.

The laboratory works aim to:

- run the three steps of the shell (`fork`, `exec`, `wait`) and see what happens when `exec` fails;
- create a zombie process on purpose and see how `wait` removes it;
- redirect the output of `ls` into a file by changing file descriptor 1 before `exec`;
- build the pipeline `ls /etc | wc -l` by hand with `pipe`, `fork` and `dup`, and see why the unused pipe ends must be closed;
- run the same `fork`/`exec`/`wait` program inside xv6 and write the program `pingpong`, which sends one byte forth and back between two processes.

THEORETICAL BACKGROUND

Creating a process: fork, exec, wait

On Unix-like systems a new program is started in three steps. `fork()` makes an almost exact copy of the calling process. It returns twice: 0 in the child and the PID of the child in the parent, or -1 on error. A function from the `exec` family (here `execlp` on Linux and `exec` in xv6) replaces the program of the current process with a new one; the PID stays the same, but the code, the data and the stack are new. If `exec` succeeds it never returns, so the code after it runs only when `exec` failed. Finally `wait()` or `waitpid()` makes the parent sleep until the child ends and gives it the exit status of the child.

Zombie processes

When a process ends, the kernel keeps a small entry for it in the process table (its PID and its exit status) until the parent reads this status with `wait`. A process that has ended but was not yet waited for is called a zombie; `ps` shows it with the letter `Z` and the mark `<defunct>`. If the parent ends without waiting, the child is adopted by the process with PID 1, which waits for it.

File descriptors and redirection

Every process has a table of open files, and a program uses them through small numbers called file descriptors: 0 is the standard input, 1 the standard output and 2 the standard error. Programs like `ls` simply write to descriptor 1 and do not know whether it is the terminal, a file or a pipe. `open()` and `dup()` always return the smallest free descriptor number, and the table of descriptors is kept by `exec`. Because of this, the shell can redirect the output of a program by closing descriptor 1 in the child and opening a file before `exec`.

Pipes

`pipe(p)` creates a buffer in the kernel with two ends: `p[0]` for reading and `p[1]` for writing. The data goes only in one direction. After `fork` both processes have both ends, so the writer moves `p[1]` to its descriptor 1 and the reader moves `p[0]` to its descriptor 0, and every process closes the ends it does not use. `read()` on an empty pipe blocks until some data comes, and it returns 0 (end of file) only when all the write ends of the pipe are closed.

xv6

xv6 is a small Unix-like teaching operating system for RISC-V, written at MIT. It runs inside the QEMU emulator and has no C library such as glibc: user programs call the system calls

of the kernel directly (`fork`, `exec`, `wait`, `pipe`, `read`, `write` and others). A new user program is a file `user/NAME.c` plus a line `$U/_NAME\` in the `UPROGS` list of the Makefile.

PART 1. FORK + EXEC + WAIT (LAB 7)

1.1 The program run.c

The work for Lab 7 was done on Fedora Linux 44 in the folder `os-lab7`, with gcc 16.2.1. The program `run.c` does what the shell does for the command `ls -l`: it copies itself with `fork`, the copy becomes `ls` with `execlp`, and the original waits for it with `waitpid`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* the three steps of the shell for the command:   ls -l   */
int main(void)
{
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);

    pid_t pid = fork();                          /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }

    if (pid == 0) {                              /* only the child runs this */
        printf("child:  my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);  /* step 2: become ls */
        perror("exec");                          /* runs only if exec failed */
        exit(127);
    }

    int status;
    waitpid(pid, &status, 0);                    /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

1.2 Running the program

The program was compiled with `gcc -Wall -Wextra` without any warnings and run. First the parent prints its PID, then the child prints its PID and the PID of its parent, then `ls -l` prints the listing, and at the end the parent prints the exit status of the child.

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 12479
child:  my PID is 12480, my parent is 12479
total 40
-rw-r--r--. 1 adelina adelina  1326 Oct 23 10:46 pipeline3.c
-rw-r--r--. 1 adelina adelina  1013 Oct 23 10:46 pipeline.c
-rw-r--r--. 1 adelina adelina   703 Oct 23 10:46 redirect.c
-rw-r--r--. 1 adelina adelina  3112 Oct 23 10:46 report.md
-rwxr-xr-x. 1 adelina adelina 12968 Oct 23 10:49 run
-rw-r--r--. 1 adelina adelina   911 Oct 23 10:42 run.c
-rw-r--r--. 1 adelina adelina   979 Oct 23 10:46 zombie.c
parent: child 12480 ended with status 0
```

Figure 1.1 – run.c with the child running ls -l

After that, the `execlp` line was changed so that the child runs `date`, and then a program that does not exist:

```
execlp("date", "date", (char *)NULL);
execlp("nosuchprog", "nosuchprog", (char *)NULL);
```

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 12488
child:  my PID is 12489, my parent is 12488
Fri Oct 23 10:49:08 AM EEST 2026
parent: child 12489 ended with status 0
```

Figure 1.2 – run.c with the child running date

```
$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 12495
child:  my PID is 12496, my parent is 12495
exec: No such file or directory
parent: child 12496 ended with status 127
```

Figure 1.3 – run.c with a program that does not exist

1.3 Answers to the questions

1. Why is perror("exec") printed only with nosuchprog?

When `execlp` succeeds, the code of `run` in the child is replaced by the code of `ls` or `date`, so the lines after `execlp` do not exist any more and are never reached. With `nosuchprog` the program cannot be found, `execlp` returns -1, and the child continues with the next line, `perror("exec")`, which prints `exec: No such file or directory`.

2. Which number does the child print as "my parent", and whose PID is it?

The child prints 12479 (Figure 1.1). It is the PID of the parent, the `./run` process, which printed the same number in the first line.

3. What status does the parent see with ls, and what status with nosuchprog?

With `ls` the status is 0, because `ls` ended successfully. With `nosuchprog` the status is 127, because after the failed exec the child calls `exit(127)`.

PART 2. A ZOMBIE, ON PURPOSE (LAB 7)

2.1 The program zombie.c

`zombie.c` is a copy of the original `run.c` (the version with `ls`) with only one new line: the parent sleeps 20 seconds before `waitpid`. The child ends almost at once, but nobody takes its exit status for 20 seconds.

```
int status;
sleep(20);                                  /* the new line */
waitpid(pid, &status, 0);                   /* step 3: wait for the child */
```

2.2 Commands executed

The program was started in the background with its output sent to `/dev/null`, and the process table was shown after 2 seconds and again after 20 more seconds. Only the lines of the shell, of `zombie` and of its child are shown in the figure.

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
$ sleep 2; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  12566    7287 Ss   bash
  12572   12566 S    ./zombie
  12574   12572 Z    [ls] <defunct>
$ sleep 20; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  12566    7287 Ss   bash
```

Figure 2.1 – The zombie child of zombie.c, and the process table 20 seconds later

2.3 Answers to the questions

1. Which letter does ls have in the STAT column, and what does it mean?

`ls` has the letter `Z`, which means zombie. It is also marked `[ls] <defunct>`: the process has already ended, but it stays in the process table because its parent (PID 12572) has not called `wait` yet.

2. After 20 seconds, is the [ls] <defunct> line still there? Which line made it disappear?

No, the line is gone, and `./zombie` itself has ended too. The line `waitpid(pid, &status, 0);` made it disappear: the parent read the exit status of the child, and then the kernel removed the child from the process table.

PART 3. REDIRECTION: LS -L > OUT.TXT (LAB 7)

3.1 The program redirect.c

`redirect.c` does what the shell does for `ls -l > out.txt`. In the child it closes descriptor 1, opens the file `out.txt`, prints to the standard error which number it got, and then runs `ls -l`. The important part is the child:

```
if (pid == 0) {
    close(1);                                   /* descriptor 1 is now free */
    int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); exit(1); }
    fprintf(stderr, "open gave descriptor %d\n", fd);
    execlp("ls", "ls", "-l", (char *)NULL);  /* ls writes to 1, as always */
    perror("exec");
    exit(127);
}
```

3.2 Commands executed

```
$ gcc -Wall -Wextra -o redirect redirect.c && ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 56
-rw-r--r--. 1 adelina adelina     0 Oct 23 10:49 out.txt
-rw-r--r--. 1 adelina adelina  1326 Oct 23 10:46 pipeline3.c
```

Figure 3.1 – redirect.c and the first lines of out.txt

Nothing from `ls` appeared on the screen. The listing is in `out.txt`, and `out.txt` itself is in the listing with size 0, because it was created by `open` before `ls` started to write in it.

3.3 Answers to the questions

1. Which number did open give, and why exactly that one?

`open` gave descriptor 1. It always returns the smallest free number; descriptor 0 (standard input) was still open, and 1 had just been closed, so 1 was the smallest free one.

2. ls was not changed at all. Why does its listing not appear on the screen?

`ls` always writes to descriptor 1. The descriptors are kept by `exec`, and in the child descriptor 1 now points to `out.txt` instead of the terminal, so the whole listing goes into the file.

3. To redirect the errors instead (like 2> err.txt), which descriptor would you close before open?

Descriptor 2, the standard error. Then `open` would return 2 and all the error messages would go into the file.

PART 4. A PIPE: LS /ETC | WC -L (LAB 7)

4.1 The program pipeline.c

`pipeline.c` builds the pipeline by hand with one pipe and two children. In the `ls` child descriptor 1 becomes the write end `p[1]`, and in the `wc` child descriptor 0 becomes the read end `p[0]`. `dup` copies a descriptor into the smallest free number, the same way as `open`. Every process closes the pipe ends it does not use.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* does the same as the shell command:   ls /etc | wc -l   */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }

    if (fork() == 0) {                          /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                              /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }

    if (fork() == 0) {                          /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                              /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }

    close(p[0]);                                /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

4.2 Commands executed

```
$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
279
$ ls /etc | wc -l
279
```

Figure 4.1 – pipeline.c compared with the real shell

Then the line `close(p[1]);` in the parent was turned into a comment, so the parent kept the write end of the pipe open:

```
close(p[0]);                                    /* the parent uses neither end */
/* close(p[1]); */
```

After this change the program did not print anything and did not end. While it was hanging, the list of its children showed that only `wc -l` was still alive; `ls` had already ended and was waited for. The program was then stopped with the `SIGINT` signal, the same signal that Ctrl-C sends.

```
$ ./pipeline &
$ ps -o pid,ppid,stat,cmd --ppid 12534
    PID    PPID STAT CMD
  12536   12534 S    wc -l
$ kill -INT 12534
```

Figure 4.2 – With p[1] open in the parent, only wc is left and never ends

4.3 Answers to the questions

1. Did ./pipeline and ls /etc | wc -l give the same number?

Yes, both printed 279 (Figure 4.1).

2. After the change, which of the two children never ends, ls or wc?

`wc` never ends. `ls` writes its listing into the pipe and ends normally.

3. Why: what is wc waiting for?

`wc` reads until end of file, and a pipe gives end of file only when all its write ends are closed, but the parent still keeps `p[1]` open, so `wc` waits forever.

4.4 Optional: three stages

The optional program `pipeline3.c` builds `ls /usr/bin | grep x | wc -l` with two pipes and three children. Every child closes all four pipe ends after its `dup`, and so does the parent. It printed the same number as the shell, 221.

```
$ gcc -Wall -Wextra -o pipeline3 pipeline3.c && ./pipeline3
221
$ ls /usr/bin | grep x | wc -l
221
```

Figure 4.3 – pipeline3.c compared with the real shell

PART 5. RUN, INSIDE XV6 (LAB 8)

5.1 Preparation

For Lab 8 the xv6 tree from `github.com/mit-pdos/xv6-riscv` was used in `~/xv6-riscv`. The RISC-V cross compiler and the emulator were installed from the Fedora packages `gcc-riscv64-linux-gnu`, `binutils-riscv64-linux-gnu` and `qemu-system-riscv`. Two lines were added at the end of the `UPROGS` list of the Makefile, each starting with a TAB:

```
UPROGS=\
    $U/_cat\
    ...
    $U/_sync\
    $U/_run\
    $U/_pingpong\
```

5.2 The program user/run.c

`user/run.c` repeats the three steps of `run.c` from Lab 7, but for xv6: there is no glibc, only the headers of the kernel, and the child becomes `echo`.

```
#include "kernel/types.h"
#include "user/user.h"

// the three steps of the shell, inside xv6:  echo hello from the child
int
main(int argc, char *argv[])
{
  printf("parent: my pid is %d\n", getpid());

  int pid = fork();                     // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }

  if(pid == 0){                         // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                 // step 2: become echo
    printf("exec failed\n");            // runs only if exec failed
    exit(1);
  }

  int status;
  wait(&status);                        // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

5.3 Running the program

xv6 was built and started with `make qemu`, and `run` was typed at the `$` prompt. To leave xv6, Ctrl-A X was used.

```
$ make qemu
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

Figure 5.1 – run inside xv6, with the child running echo

Then the child was changed to run `ls`, and after that a program that does not exist:

```
char *args[] = { "ls", 0 };
exec("ls", args);                  // step 2: become ls

exec("nosuchprog", args);
```

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35944
echo           2 4 34840
forktest       2 5 17776
grep           2 6 43368
init           2 7 35192
kill           2 8 34744
ln             2 9 34536
ls             2 10 42024
mkdir          2 11 34792
rm             2 12 34776
sh             2 13 57152
stressfs       2 14 35616
usertests      2 15 201600
grind          2 16 50416
wc             2 17 36912
zombie         2 18 34088
logstress      2 19 36760
forphan        2 20 35552
dorphan        2 21 35016
sync           2 22 34192
run            2 23 35104
pingpong       2 24 35104
console        3 25 0
parent: child 4 ended with status 0
```

Figure 5.2 – run inside xv6, with the child running ls

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

Figure 5.3 – run inside xv6, with a program that does not exist

5.4 Answers to the questions

1. The three calls of run.c in xv6 and on Linux. What is the same?

Step	Linux (Lab 7)	xv6 (Lab 8)
copy the process	`fork()`	`fork()`
become another program	`execlp("ls", "ls", "-l", NULL)`	`exec("echo", args)`
wait for the child	`waitpid(pid, &status, 0)`	`wait(&status)`

The three steps and their order are the same: `fork` copies the process, `exec` turns the child into another program and returns only on error, and the parent waits for the child and gets its exit status. `fork` is exactly the same; only `exec` and `wait` have slightly different names and arguments, because glibc on Linux gives more variants of them.

2. Who prints "hello from the child": the code of run, or echo?

`echo` prints it. After `exec` the child is no longer `run`, it is `echo`, and "hello from the child" are just the arguments that `run` gave to `echo` in `args`.

3. Why is the status 1 here and 127 on Linux?

The status is the number that the child gives to `exit`. After the failed exec, the xv6 program calls `exit(1)`, while the Linux program in Lab 7 called `exit(127)`. The parent just shows this number.

PART 6. PINGPONG (LAB 8)

6.1 The program user/pingpong.c

The parent sends one byte to the child through the pipe `p2c`. The child reads it, prints `<pid>: received ping`, writes the byte back through the second pipe `c2p` and exits. The parent reads the byte, prints `<pid>: received pong` and waits for the child. The four TODO lines were completed with `read(fd, &b, 1)` and `write(fd, &b, 1)`.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];     // parent-to-child, child-to-parent
  char b;

  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    read(p2c[0], &b, 1);                // read 1 byte from p2c[0] into b
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);               // write the byte b into c2p[1]
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  b = 'x';
  write(p2c[1], &b, 1);                 // write 1 byte into p2c[1]
  read(c2p[0], &b, 1);                  // read 1 byte from c2p[0] into b
  printf("%d: received pong\n", getpid());
  wait(0);
  exit(0);
}
```

6.2 Running the program

```
$ pingpong
6: received ping
5: received pong
```

Figure 6.1 – pingpong inside xv6

The child (PID 6) prints "received ping" first, and only after that the parent (PID 5) prints "received pong".

6.3 Answers to the questions

1. Why do you need TWO pipes, and not one for both directions?

A pipe carries data only in one direction, from the write end to the read end. If one pipe were used for both directions, the parent could read back its own byte before the child takes it, and the two processes could not know whose byte is in the pipe. So one pipe is used for parent to child and the other for child to parent.

2. Which call makes the parent wait until the child has replied?

`read(c2p[0], &b, 1)`. When the pipe is empty, `read` blocks, so the parent stays in this call until the child writes the byte into `c2p[1]`. In the same way the child waits in its `read` until the parent sends the ping.

CONCLUSION

In these two laboratory works the creation of processes, the redirection of output and pipes were studied on Linux and inside xv6.

In Lab 7, `run.c` showed the three steps of the shell: `fork` makes a copy, `exec` turns the child into another program and never returns if it works, and `waitpid` gives the parent the status of the child, 0 for `ls` and 127 when the program does not exist. `zombie.c` showed that a child that ended stays in the process table as `Z <defunct>` until its parent calls `wait`. `redirect.c` and `pipeline.c` showed that redirection and pipes are done only by changing descriptors 0 and 1 before `exec`, using the rule that `open` and `dup` take the smallest free number; `ls` and `wc` were not changed at all. Leaving one write end of the pipe open was enough to make `wc` wait forever.

In Lab 8, the same program ran inside xv6 with `fork`, `exec` and `wait`, with the same behaviour as on Linux; only the exit code after a failed exec was different (1 instead of 127). The program `pingpong` showed that two processes can talk through two pipes, one for each direction, and that a blocking `read` is enough to make one process wait for the other.

These system calls are the basis of every Unix-like shell, and xv6 shows that a small kernel needs only a few of them to start programs and connect them together.

REFERENCES

1. Cox R., Kaashoek F., Morris R. xv6: a simple, Unix-like teaching operating system, chapter 1 "Operating system interfaces". MIT, 2024. [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/xv6/book-riscv-rev4.pdf
2. mit-pdos/xv6-riscv – source code. [online]. Available: https://github.com/mit-pdos/xv6-riscv
3. MIT 6.1810 Operating System Engineering. Lab: Xv6 and Unix utilities (pingpong). [online]. Available: https://pdos.csail.mit.edu/6.1810/2024/labs/util.html
4. Linux manual pages: fork(2), execve(2), wait(2), pipe(2), dup(2), open(2). [online]. Available: https://man7.org/linux/man-pages/
5. Operating Systems course, FCIM/FAF, UTM, 2026–2027. Lab 7: Three Small Programs: Processes, Redirection, a Pipe.
6. Operating Systems course, FCIM/FAF, UTM, 2026–2027. Lab 8: The Same Calls, Inside Your Own OS.

Github repository: https://github.com/Adelina1905/os-2026-AdelinaTemciuc